ACME Salary Management System

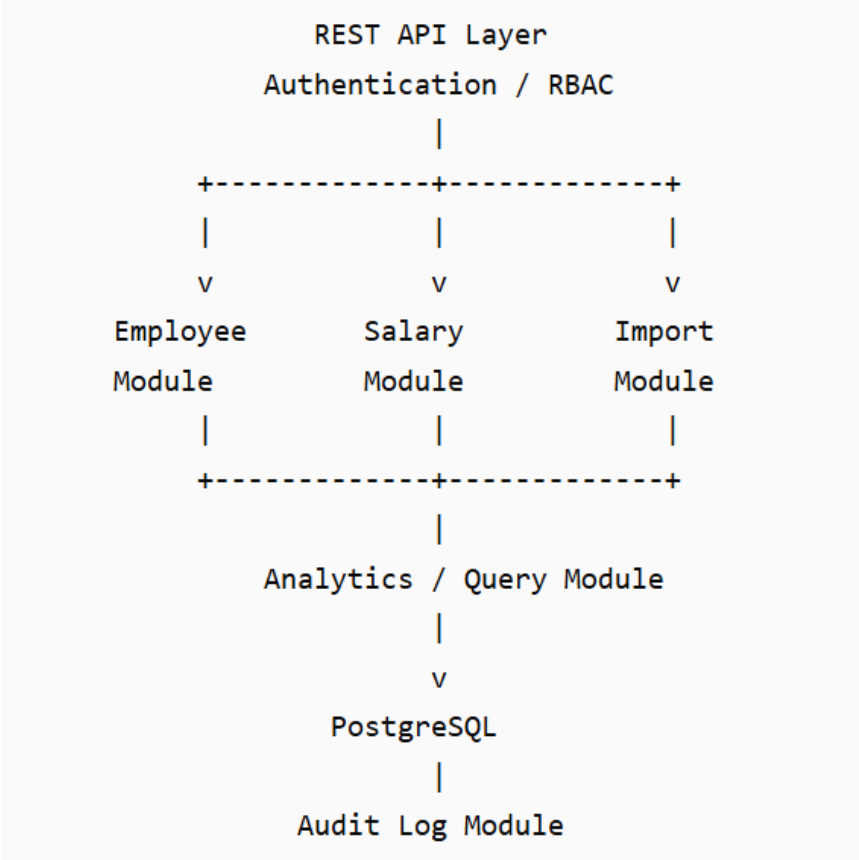
Backend Architecture, Database Schema & API Specification

Version: 1.0
Status: Proposed design
Database: PostgreSQL
API style: REST / JSON
Architecture: Modular monolith

---

1. Architecture Overview

1.1 High-Level Architecture



1.2 Backend Modules

Module	Responsibilities
Authentication & Authorization	Login, user identity, roles, access control
Employee Management	Employee CRUD, search, filtering, pagination
Compensation Management	Salary records, effective dates, salary history
Data Import	CSV/XLSX parsing, validation, duplicate detection, import results
Analytics	Salary statistics, country/department breakdowns, reports
Question Answering	Translate supported questions into validated analytics queries
Audit Logging	Track salary changes and sensitive administrative actions

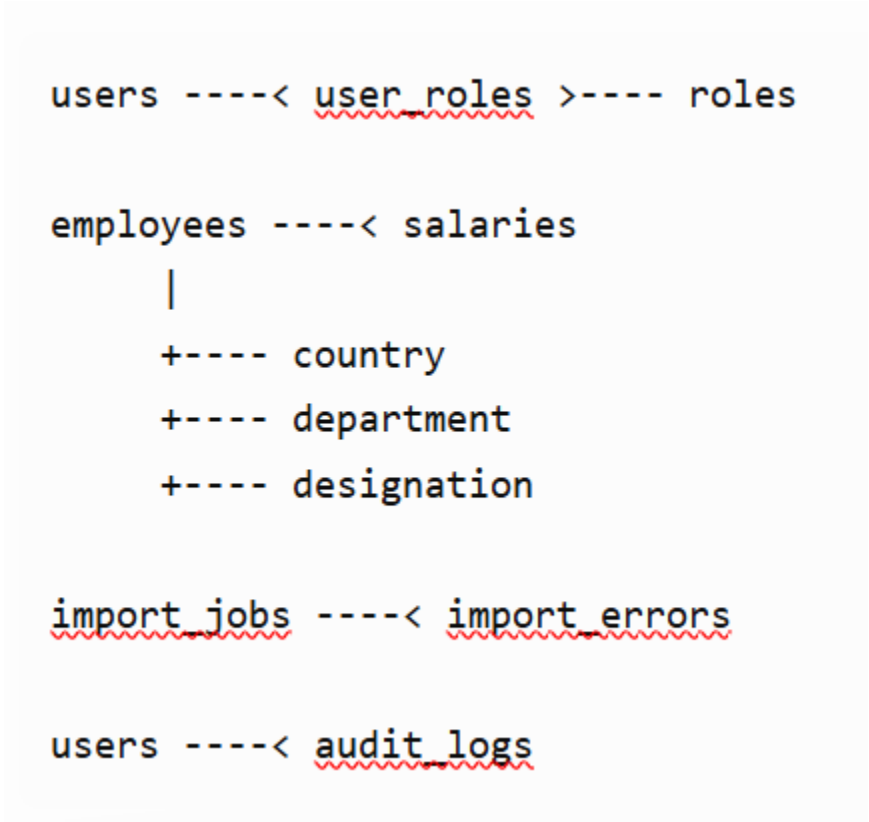
All modules run within one deployable backend application. They share a PostgreSQL database but should maintain clear service boundaries.

Recommended stack: Java 21 with Spring Boot, Spring Security, Spring Data JPA, Flyway, and PostgreSQL. Python with FastAPI is also viable if preferred by the development team.

2. Database Design

Use normalized relational tables for employee identity, salary history, organizational attributes, and auditability. Store monetary values using `NUMERIC`, never floating-point types.

2.1 Entity Relationships



Countries, departments, and designations are reference entities. Employee identity is independent of salary history, so compensation changes do not overwrite employee records.

2.2 Core PostgreSQL Schemas

A. `countries`

Column	Type	Constraints
code	CHAR(2)	Primary key; ISO 3166-1 alpha-2
name	VARCHAR(100)	Not null, unique
default_currency	CHAR(3)	ISO 4217 currency code

B. `departments`

Column	Type	Constraints
id	BIGINT	Primary key
name	VARCHAR(100)	Not null, unique
created_at	TIMESTAMPTZ	Not null

C. `designations`

Column	Type	Constraints
id	BIGINT	Primary key
title	VARCHAR(150)	Not null, unique

D. `employees`

Column	Type	Constraints
id	UUID	Primary key
employee_number	VARCHAR(50)	Not null, unique
first_name	VARCHAR(100)	Not null
last_name	VARCHAR(100)	Not null
email	VARCHAR(255)	Unique; nullable if not required
country_code	CHAR(2)	Foreign key to countries
department_id	BIGINT	Foreign key to departments

designation_id	BIGINT	Foreign key to designations
employment_status	VARCHAR(20)	ACTIVE, INACTIVE, TERMINATED
hire_date	DATE	Nullable
created_at	TIMESTAMPTZ	Not null
updated_at	TIMESTAMPTZ	Not null

Use UUIDs as internal identifiers and a separate human-readable employee number for business operations. Deactivate employees instead of deleting records referenced by salary history.

E. `salaries`

Each row represents a salary record for a specific effective period.

Column	Type	Constraints
id	UUID	Primary key
employee_id	UUID	Foreign key to employees
annual_amount	NUMERIC(19,4)	Not null, greater than zero

currency_code	CHAR(3)	Not null; ISO 4217
effective_from	DATE	Not null
effective_to	DATE	Nullable; exclusive upper bound
change_reason	VARCHAR(255)	Nullable
created_by	UUID	Foreign key to users
created_at	TIMESTAMPTZ	Not null

Salary history rules:

- Use effective-date ranges of `[effective_from, effective_to)`.
- A null `effective_to` means the record has no scheduled end date.
- Prevent overlapping effective periods for the same employee, using a PostgreSQL exclusion constraint or equivalent transactional validation.
- Retain previous salary records when compensation changes.
- Store the annualized base salary consistently; model bonuses and other compensation components separately if they become in scope.

F. `users` and `roles`

Table	Important columns
users	`id UUID PK`, `email`, `password_hash` or external identity subject, `is_active`, `created_at`

roles	`id BIGINT PK`, `name UNIQUE`
user_roles	`user_id FK`, `role_id FK`, composite primary key

Start with the `HR_MANAGER` role. Passwords must never be stored in plaintext. Prefer a managed identity provider or securely hashed credentials.

G. `audit_logs`

Column	Type	Constraints
id	BIGINT	Primary key
actor_user_id	UUID	Foreign key to users
action	VARCHAR(50)	Not null
entity_type	VARCHAR(50)	Not null
entity_id	VARCHAR(100)	Not null
before_data	JSONB	Nullable
after_data	JSONB	Nullable

occurred_at	TIMESTAMPTZ	Not null
request_id	UUID	Nullable

Audit salary creation, amendments, employee deactivation, imports, and administrative actions. Restrict audit access and avoid logging credentials or unnecessary sensitive data.

H. `import_jobs` and `import_errors`

`import_jobs`

`id UUID PK`, `uploaded_by UUID FK`, `file_name`, `status`, `total_rows`, `successful_rows`, `failed_rows`, `created_at`, `completed_at`.

`import_errors`

`id BIGINT PK`, `import_job_id UUID FK`, `row_number`, `field_name`, `error_code`, `message`.

Support statuses such as `PENDING`, `VALIDATING`, `COMPLETED`, `COMPLETED_WITH_ERRORS`, and `FAILED`. Store the original file only if required, with access controls and retention limits.

2.3 Indexing Strategy

Create indexes on:

- `employees(employee_number)` — unique.
- `employees(country_code, department_id, employment_status)`.
- `employees(designation_id)`.
- `salaries(employee_id, effective_from DESC)`.
- `salaries(currency_code)`.
- `audit_logs(entity_type, entity_id, occurred_at DESC)`.
- `import_errors(import_job_id, row_number)`.

Use server-side pagination and parameterized queries. Review execution plans and add indexes based on actual report and search patterns.

---

3. REST API Specification

Base path: `/api/v1`

All endpoints return JSON. Protected endpoints require an authenticated session or bearer token, according to the chosen authentication mechanism.

3.1 Authentication

Method	Endpoint	Purpose
POST	`/auth/login`	Authenticate HR manager
POST	`/auth/logout`	End session, where applicable
GET	`/auth/me`	Return authenticated user and roles

If using an external identity provider, delegate login and token issuance to that provider rather than implementing password management unnecessarily.

3.2 Employee APIs

Method	Endpoint	Purpose
POST	`/employees`	Create employee
GET	`/employees`	List, search, filter, and paginate

GET	`/employees/{id}`	Retrieve employee details
PATCH	`/employees/{id}`	Update permitted employee fields
POST	`/employees/{id}/deactivate`	Deactivate employee

Supported list query parameters:

`search`, `country`, `department_id`, `designation_id`, `status`, `page`, `size`, `sort`.

Example:

```
GET /api/v1/employees?country=IN&department_id=2&page=0&size=25
```

3.3 Salary APIs

Method	Endpoint	Purpose
POST	`/employees/{id}/salaries`	Create salary record
GET	`/employees/{id}/salaries`	Retrieve salary history
GET	`/salaries/{id}`	Retrieve salary record
PATCH	`/salaries/{id}`	Correct permitted fields under audit

GET	`/salaries/current`	Query current salary records across employees

Salary changes should create a new effective-dated record whenever possible. Historical corrections require explicit authorization, validation, and audit logging.

Example request:

```
{
  "annual_amount": 85000.00,
  "currency_code": "USD",
  "effective_from": "2026-11-01",
  "change_reason": "Annual compensation review"
}
```

The backend validates the currency, amount, employee existence, effective dates, and overlapping periods before committing the change.

3.4 Import APIs

Method	Endpoint	Purpose
POST	`/imports`	Upload CSV/XLSX and create import job
GET	`/imports/{jobId}`	Retrieve status and row counts
GET	`/imports/{jobId}/errors`	Retrieve validation errors

POST	`/imports/{jobId}/commit`	Commit validated data, if using staged imports

For larger imports, process jobs asynchronously. Validate and stage records before committing. Use transactional batches, explicit duplicate-handling rules, and an import idempotency strategy to avoid accidental duplicate records.

3.5 Analytics APIs

Method	Endpoint	Purpose
GET	`/analytics/overview`	Headcount and salary summary
GET	`/analytics/salaries/by-country`	Country-level salary statistics
GET	`/analytics/salaries/by-department`	Department-level statistics
GET	`/analytics/salaries/distribution`	Salary ranges and distribution
GET	`/analytics/highest-paid`	Top-paid employees
GET	`/analytics/salaries/export`	Export authorized filtered results

Common filters include `country`, `department_id`, `designation_id`, `currency_code`, and `as_of_date`.

For each report, explicitly define whether the metric represents annual base salary, total compensation, active employees, or a historical snapshot.

Do not combine different currencies into a single total or average without an explicit conversion policy.

3.6 Question-Answering API

Method	Endpoint	Purpose
POST	`/questions`	Answer a supported compensation question

Example request:


```
{
  "question": "What is the average annual salary in India?",
  "as_of_date": "2026-11-01"
}
```


Example response:


```
{
  "question": "What is the average annual salary in India?",
  "status": "answered",
  "metric": "average_annual_base_salary",
  "filters": {
    "country": "IN",
    "as_of_date": "2026-11-01"
  },
  "currency_code": "INR",
  "value": 1250000.00,
```

"employee_count": 420,

"explanation": "Average annual base salary for employees with a salary effective on the specified date."

}

The values above are illustrative, not actual ACME data.

Initially, map questions to a defined set of analytics operations. If an LLM is introduced, it must select from approved query templates or a constrained query representation. Never execute arbitrary generated SQL, allow model output to bypass authorization, or expose employee-level salary information to unauthorized users.

3.7 Audit APIs

Method	Endpoint	Purpose
GET	`/audit-logs`	Search authorized audit events
GET	`/audit-logs/{id}`	Retrieve an individual event

Restrict these endpoints to authorized administrators or auditors. Record the actor, timestamp, affected entity, and change details for sensitive operations.

---

4. API Conventions and Error Handling

Standard response behavior

- `200 OK` — successful read or update.
- `201 Created` — successful resource creation.
- `202 Accepted` — asynchronous import accepted.
- `400 Bad Request` — malformed request.
- `401 Unauthorized` — authentication required or invalid.
- `403 Forbidden` — insufficient permissions.

- `404 Not Found` — resource does not exist or is not accessible.
- `409 Conflict` — duplicate employee number or conflicting salary period.
- `422 Unprocessable Content` — valid JSON but invalid business data.
- `429 Too Many Requests` — rate limit exceeded.
- `500 Internal Server Error` — unexpected failure.

Use a consistent error body containing `code`, `message`, `field_errors` when applicable, and `request_id`. Never return stack traces or sensitive database details to clients.

Definition of done: HR managers can securely manage employee and salary data, migrate spreadsheet records with actionable validation feedback, retrieve accurate filtered reports, and ask supported compensation questions with reproducible results and an auditable history.